

Digital controller for bionic robotic arm

Theoretical background

Space exploration and planetary research largely rely on the analysis of rocks, which serve as invaluable "archives" of a planet's geological history. They provide information about the evolution of celestial bodies, geological processes, and ancient environments, which is crucial for understanding the universe. Furthermore, rock analysis helps identify locations where liquid water might have existed - a necessary condition for the search for extraterrestrial life. Rocks can also contain valuable minerals and chemical compounds, representing potential resources for future space missions, such as for producing fuel or oxygen.

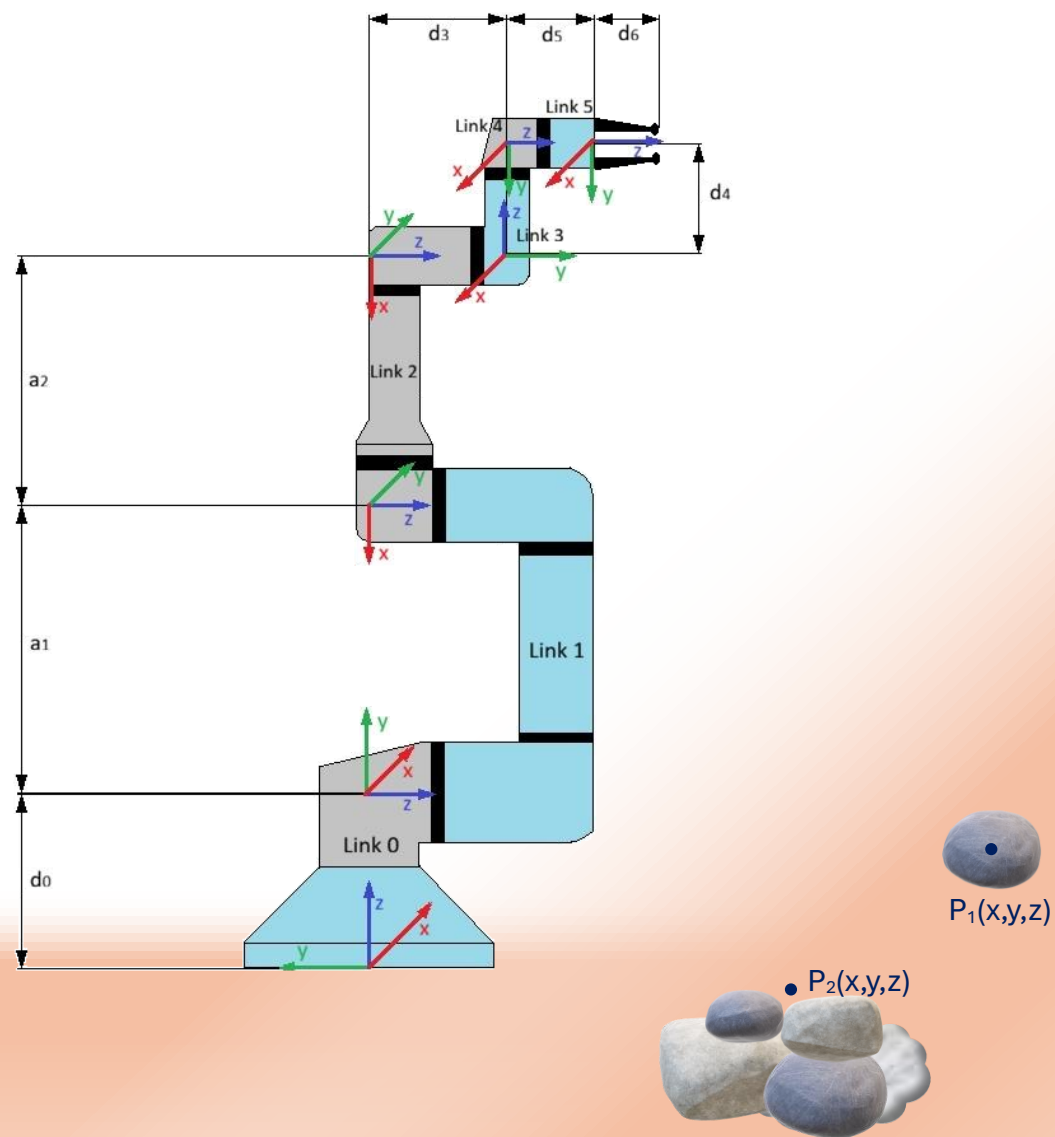
To effectively conduct these studies in the challenging conditions of space, precise robot control is essential. This is where inverse kinematics plays a crucial role. It is a fundamental problem in robotics that involves determining the joint angles of a robot so that its end-effector (e.g., a gripper or an analytical tool) reaches a specified position and orientation in space. Thanks to inverse kinematics, precise robot control is possible, enabling it to plan movements, grasp objects - such as rock samples - and interact with its environment. In this way, advanced robotic algorithms facilitate the remote execution of complex scientific and logistical operations that are fundamental to our knowledge of the cosmos.

Goal

Create an RTL to solve the inverse kinematics problem for six degrees of freedom robot manipulator. Based on the supplied drawing, Denavit-Hartenberg table, and inverse kinematics equations, calculate the joint angles ($\varphi_1, \varphi_2, \varphi_3, \varphi_4$) required to move the robot's end-effector from the starting point (P0) to point P1 (picking up a stone) and then to point P2 (placing the stone on a pile). Two of the angle φ_5, φ_6 have constant values.

Equations for forward kinematics of given robot arm are:

- $endx = 0,17415\sin(\varphi^1) + 0,11985\sin(\varphi^2 + \varphi^3 + \varphi^4)\cos(\varphi^1) - 0,00005\cos(\varphi^1)(11431,0\cos(\varphi^2 + \varphi^3) + 12254,0\cos(\varphi^2)) + 0,32655 \cos(\varphi_2 + \varphi_3 + \varphi_4) \cos(\varphi_1)$
- $endy = 0,11985 \sin(\varphi_2 + \varphi_3 + \varphi_4) \sin(\varphi_1) - 0,17415 \cos(\varphi_1) - 0,00005 \sin(\varphi_1) (11431,0 \cos(\varphi_2 + \varphi_3) + 12254,0 \cos(\varphi_2)) + 0,32655 \cos(\varphi_2 + \varphi_3 + \varphi_4) \sin(\varphi_1)$
- $endz = 0,1807 - 0,57155 \sin(\varphi_2 + \varphi_3) - 0,6127 \sin(\varphi_2) + 0,32655 \sin(\varphi_2 + \varphi_3 + \varphi_4) - 0,11985 \cos(\varphi_2 + \varphi_3 + \varphi_4)$
- $r_{33} = \sin(\varphi_2 + \varphi_3 + \varphi_4)$



The Denavit-Hartenberg Table	a	α	d	θ
1	0	90°	0,18070	Φ_1
2	-0,61270	0°	0	Φ_2
3	-0,57155	0°	0	Φ_3
4	0	90°	0,17415	Φ_4
5	0	-90°	0,11985	-90°
6	0	0°	0,11655	180°
7	0	0°	0,21000	0°

Provided module

The task_7.sv module is the top module of your design. If you wish, you may add other files for supportive submodules. Inputs and outputs are declared as:

Signal name	Signal type	Bit length
i_clk	input	1 bit
i_rst	input	1 bit
i_first	input	1 bit
i_valid	input	1 bit
i_data	input	32 bits
i_last	input	1 bit
o_valid	output	1 bit
o_data	output	32 bits
o_last	output	1 bit

The reset is synchronous and active high.

Input and output interfaces

The task receives one input packet containing three target points. Each point is encoded as one header field identifying the point (32'b00 for the initial position, 32'b01 for p1, 32'b11 for p2), followed by four 32-bit input words: the end-effector target coordinates x, y, z, and the orientation matrix element r33. The data is in Q20.12 fixed-point format. The first word of the packet is marked with i_first, and the last word of the packet is marked with i_last.

For each input point, the task should compute the corresponding joint angles phi1, phi2, phi3, and phi4, and transmit one output packet containing the original headers together with the four corresponding 32-bit output words. During transmission, o_valid shall be asserted in cycle alignment with o_data, and the last word of the output packet shall be marked with o_last.

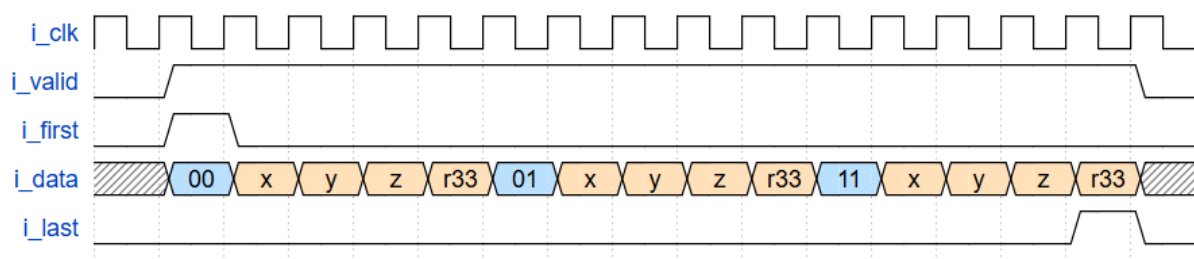


Figure 1 Input waveform

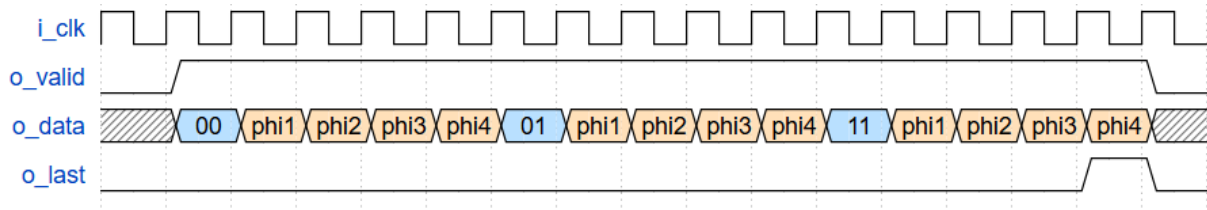
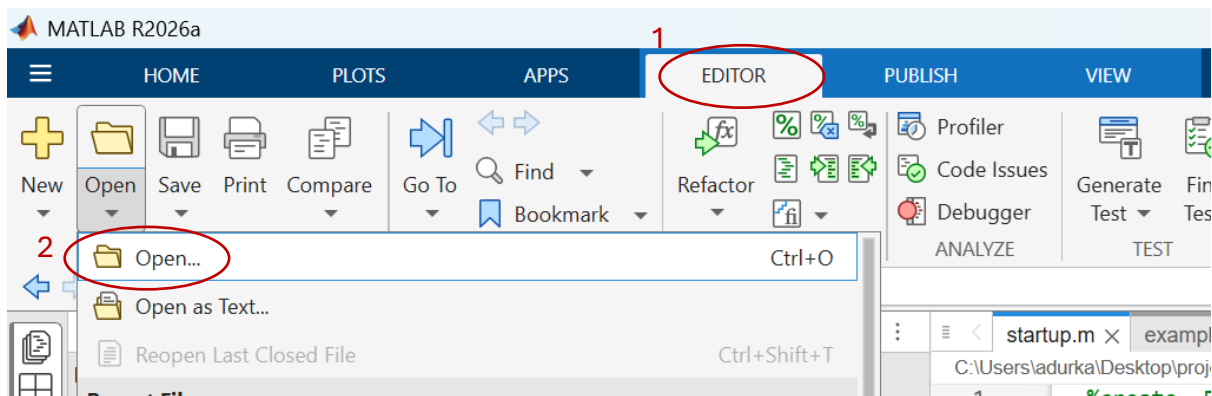


Figure 2 Output waveform

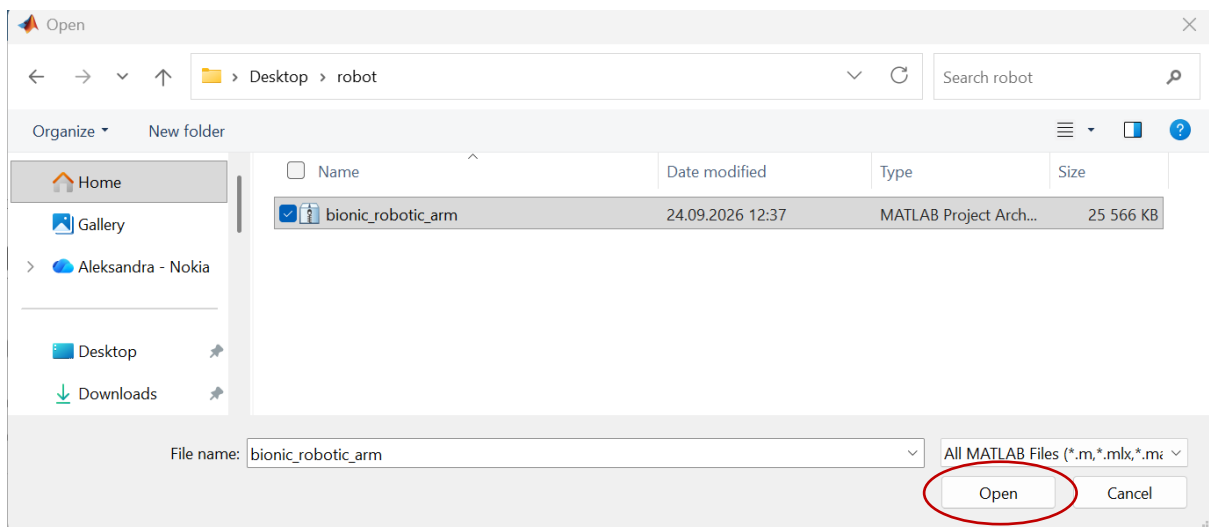
In order to visualize the motion of the robot arm in Simulink (You can open it by downloading the **bionic_robotic_arm.mlproj** file and opening it in MATLAB), the angles must be normalized correctly. It should look like this:

- $0 \leq \varphi_1 \leq 360$
- $-360 \leq \varphi_2 \leq 0$
- $0 \leq \varphi_3 \leq 360$
- $0 \leq \varphi_4 \leq 360$

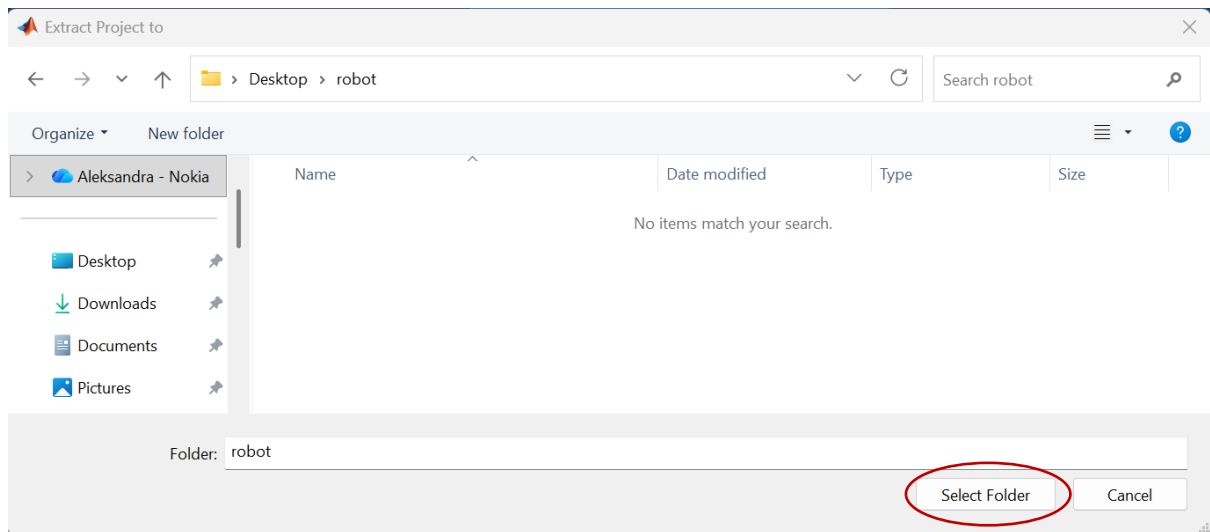
Once you have downloaded the file, move it to a new folder. Next, open MATLAB and, in the 'Editor' tab, select the 'Open' option:



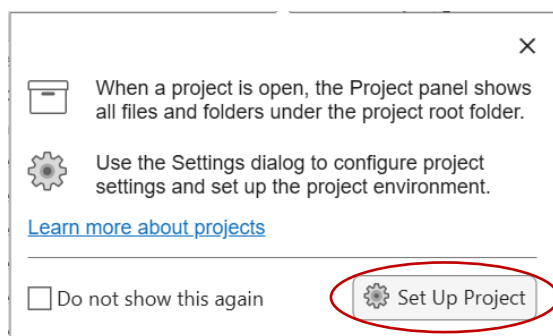
Then locate the folder containing the file, select this file and click 'Open' button:



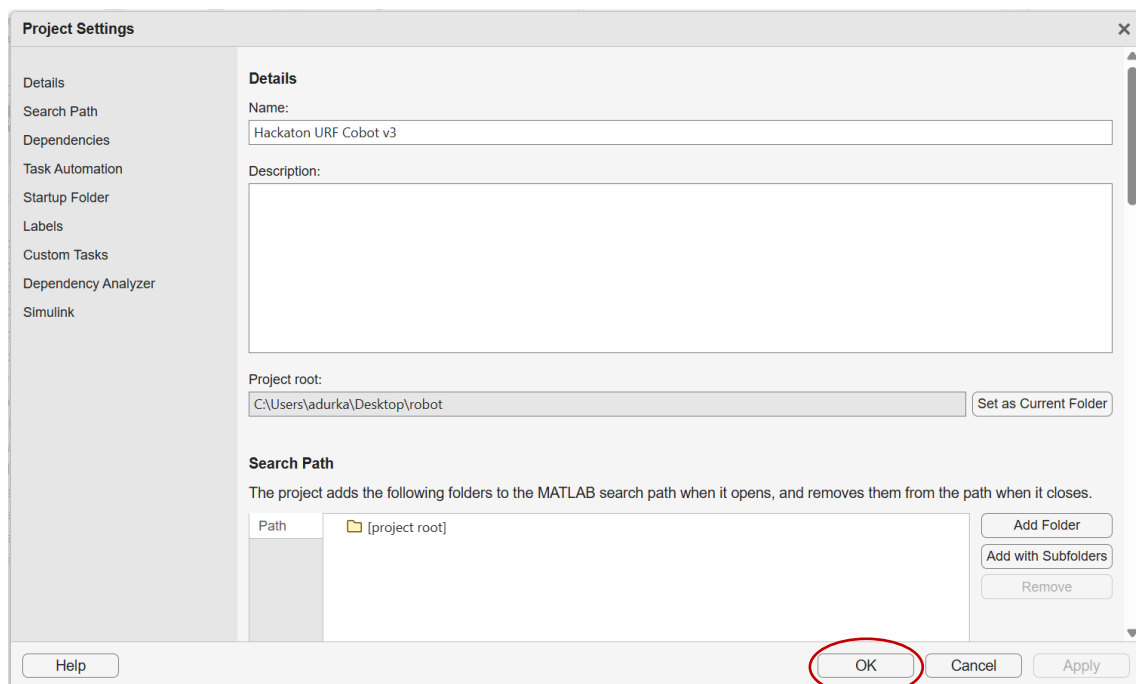
Wait until the 'Extract Project to' window opens, and after locating your folder again (if necessary), click the 'Select Folder' button.



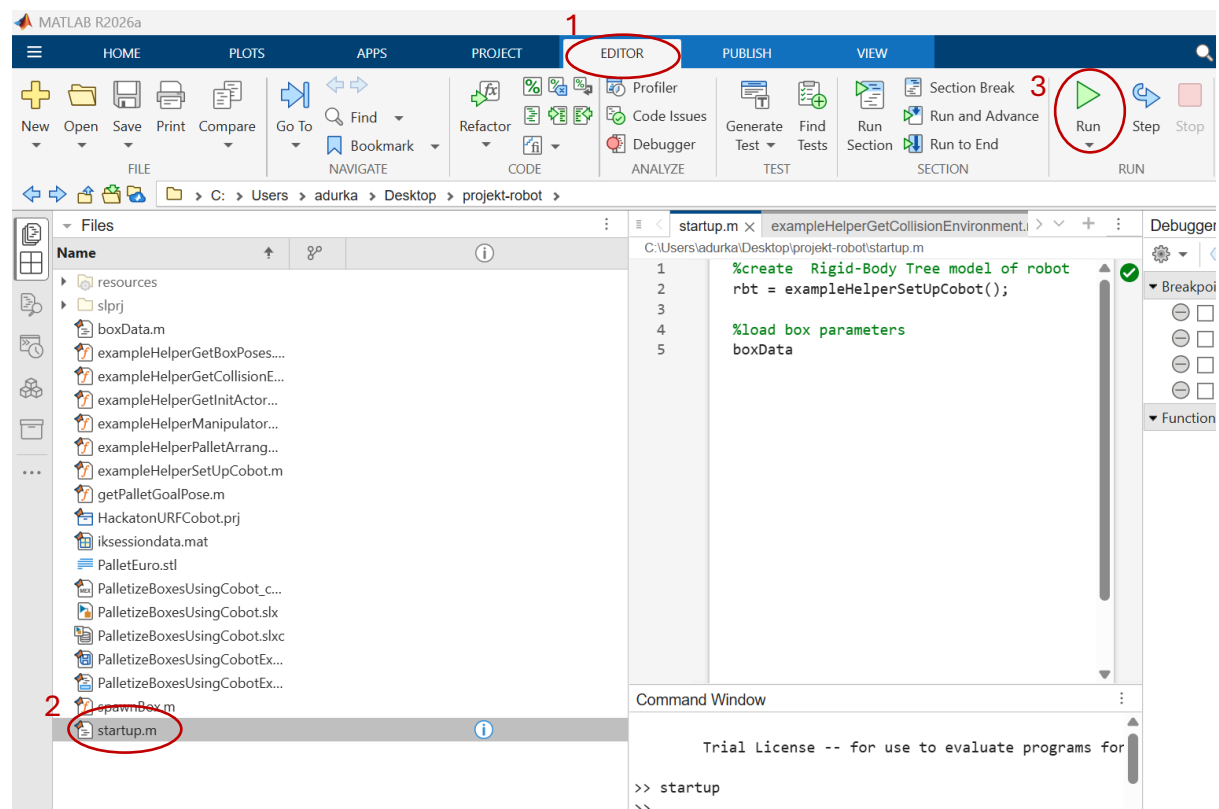
If a window like the one below appears, select the 'Set Up Project' option:



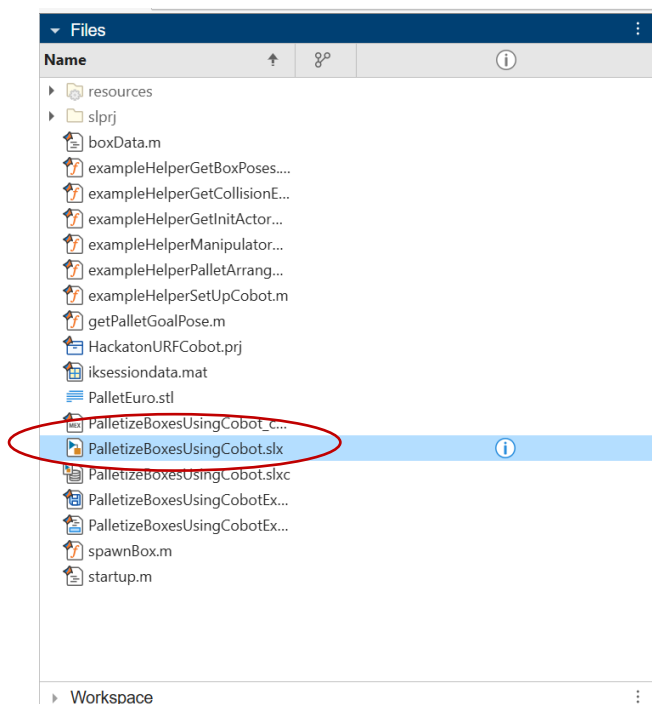
Then, in the 'Project Settings' window, click 'OK' without changing anything:



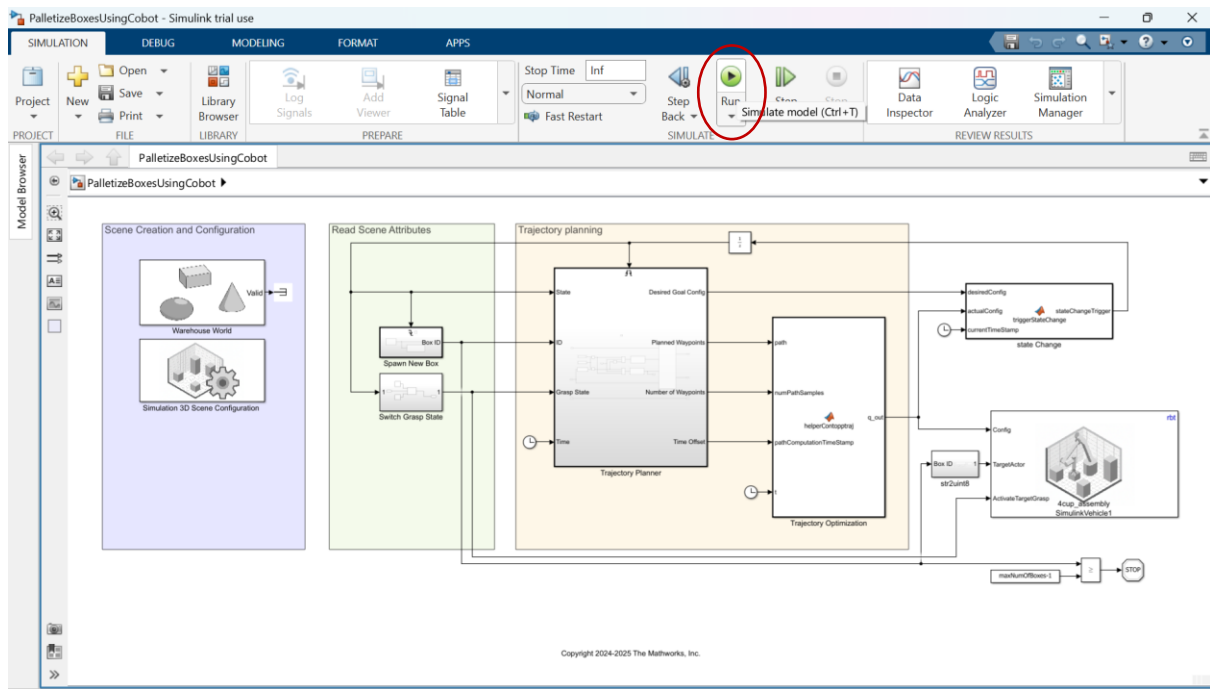
Next, go to the 'Editor' tab, locate the **startup.m** file in the 'Files' window and select it, then press the Run button.



Then select the **PalletizeBoxesUsingCobot.slx** file and double-click it:



Wait until the model in Simulink has opened correctly, then start the simulation by pressing 'Run':



Evaluating the task

For the task to be considered complete, all four data sets for each target position, each containing four angle values, must be numerically accurate in relation to the reference output values, which are calculated using the equations in the “Goal” section.

In Development mode, the task will be tested using a single set of data. In Evaluation mode, the task will be tested using 4 fixed sets of data, which remain the same across all runs and are identical for every team (for more information, check chapter 2.3.1 **Testing modes** in the **FPGA_Hackathon_2026_Project_Guide** document).

You earn four points for correctly processing each test vector. In Development mode, this simply indicates that the task has passed the test. In Evaluation mode, your total base score is calculated by multiplying 4 by the number of test vectors, meaning you can earn up to 16 points.

Five additional points may be awarded for **resource utilization if latency did not exceed 10 μ s** (for more information, check chapter 2.3.2 **Bonus Points** in the **FPGA_Hackathon_2026_-Project_Guide** document).